

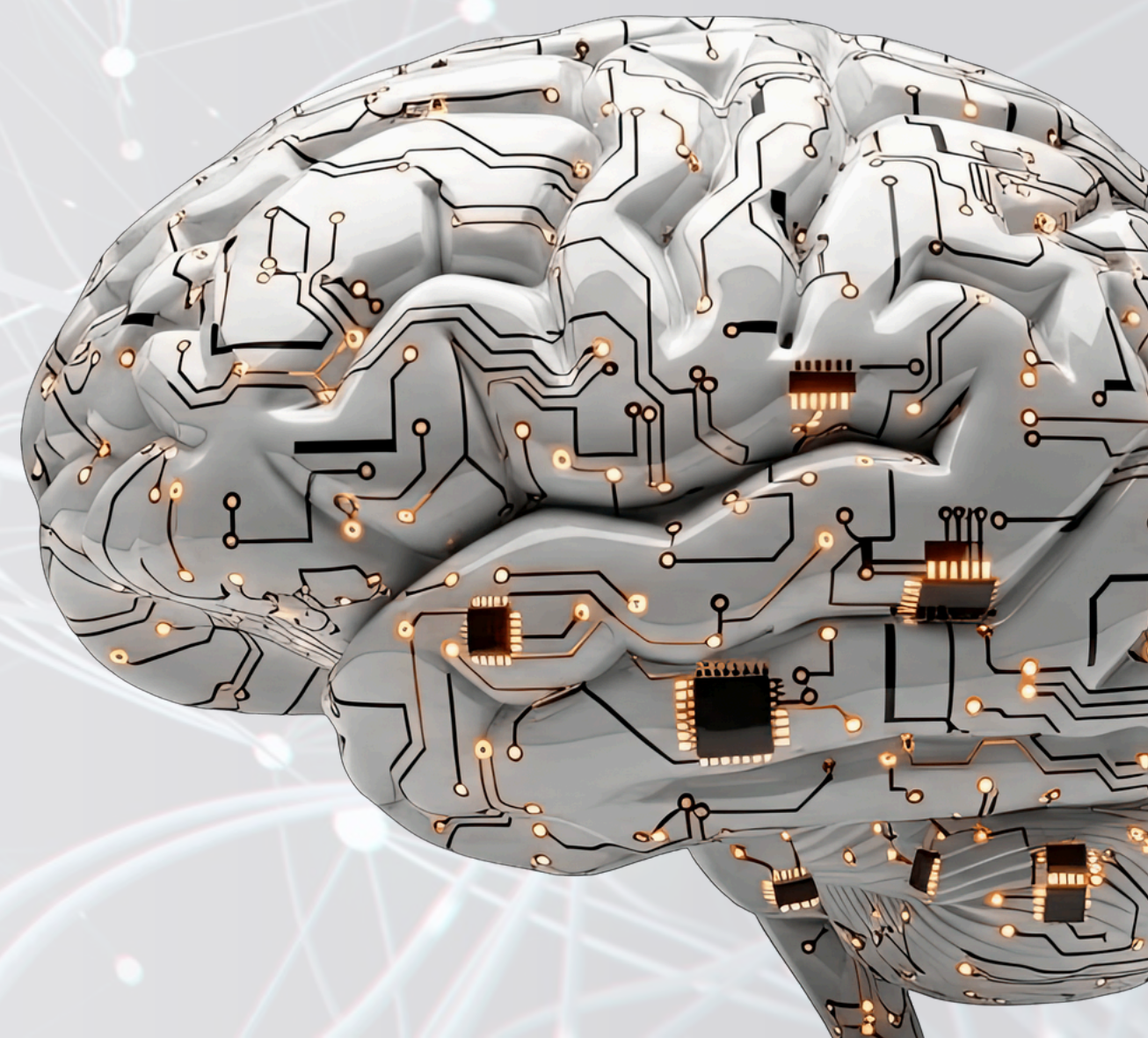
STRUCTURE OVER PROMPTS

Building SRE Agents on Gemini Enterprise Agent Platform

Olga Mirensky

Senior Staff SRE, Ping Identity

DevFest, Melbourne, 3 Oct 2026





AGENDA

Introduction – GEAP and Ping Identity

What is Gemini Enterprise Agent Platform. What is Ping Identity. What challenges we are working with and how Agentic SRE Platform helps us solve them.

Google AI SRE

Google's blueprint for Agentic SRE

ADK and Agent Runtime

Technical showcase of key ADK capabilities and optimisations. How agent runtime implements these constructs

Demo

Sample e2e-agent working on an OSS repo, analysing test failures, storing them in BigQuery enabling environment-wide intelligence. From agent Playground to wider GEAP ecosystem.

Introducing Gemini Enterprise Agent Platform, powering the next wave of agents

April 23, 2026

What's new: Today, we're launching Gemini Enterprise Agent Platform — our new, comprehensive platform to build, scale, govern, and optimize agents. It's the evolution of Vertex AI, bringing the model selection, model building, and agent building capabilities that customers love, together with new features for agent integration, DevOps, orchestration, and security.

✦ Gemini Enterprise Agent Platform

Build

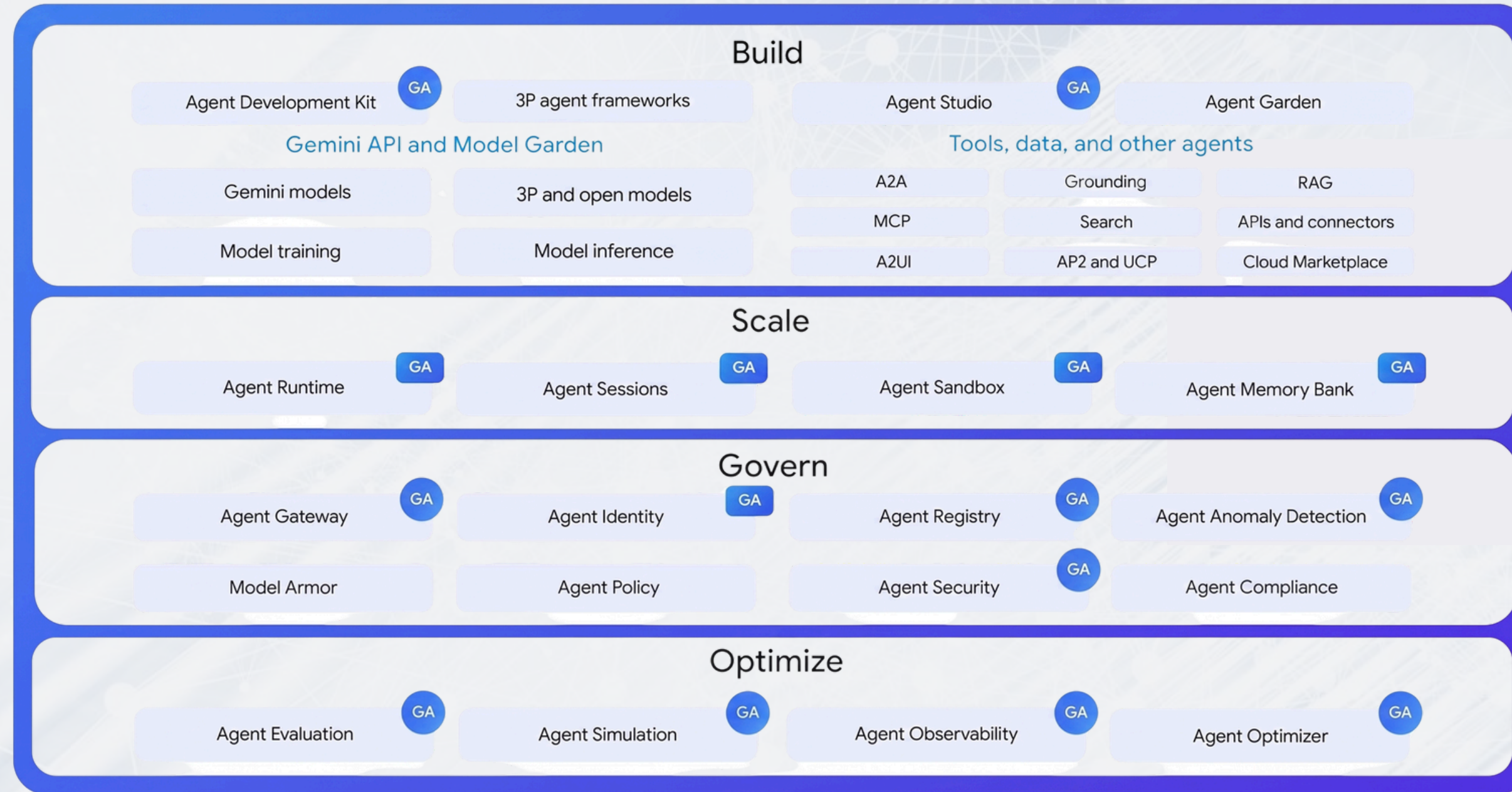
Scale

Govern

Optimize

<https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform>

◆ Gemini Enterprise Agent Platform



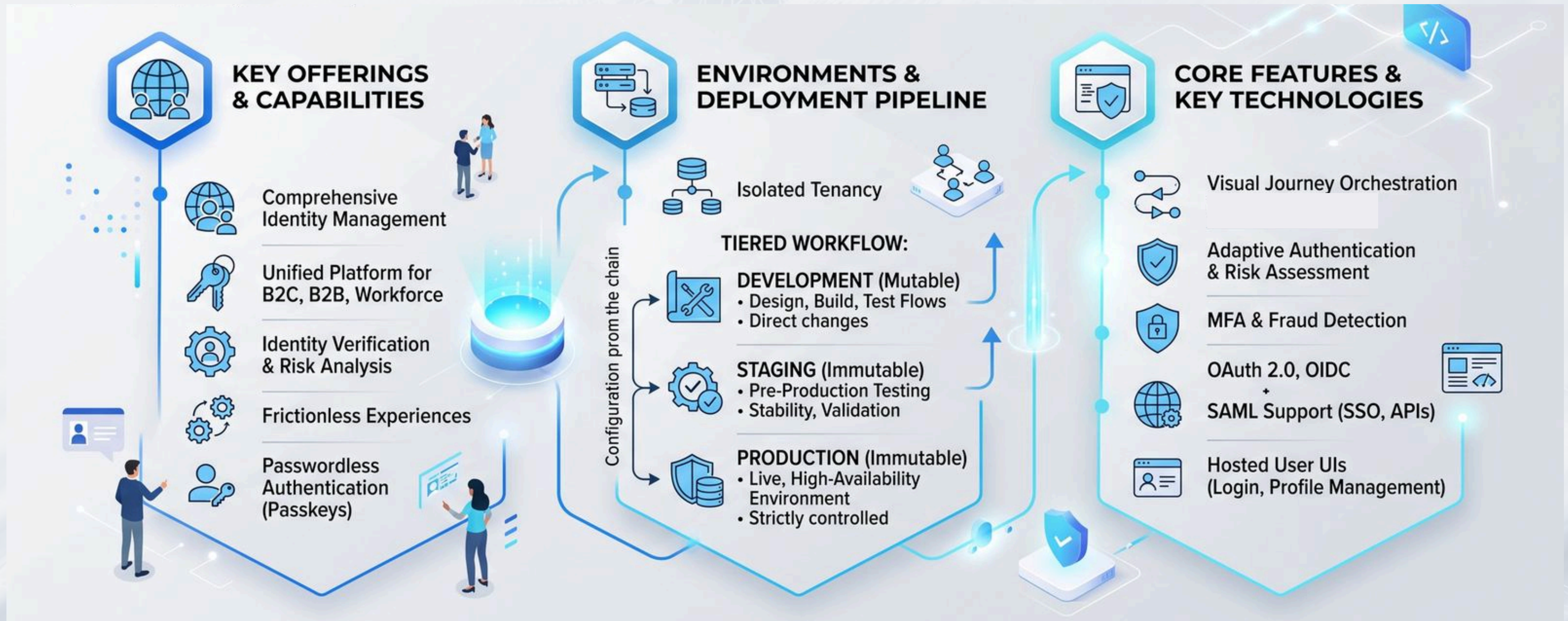
GA GA at announcement
Apr, 2026

GA GA as of Sept 2026

<https://cloud.google.com/blog/products/ai-machine-learning/introducing-gemini-enterprise-agent-platform>

<https://cloud.google.com/blog/products/ai-machine-learning/whats-new-in-gemini-enterprise-agent-platform>

PingOne Advanced Identity Cloud



Ping Identity AGW Partner

Agent Gateway acts as a programmable data plane and enforcement point for AI agent traffic - LLM requests, MCP calls, tool interactions.

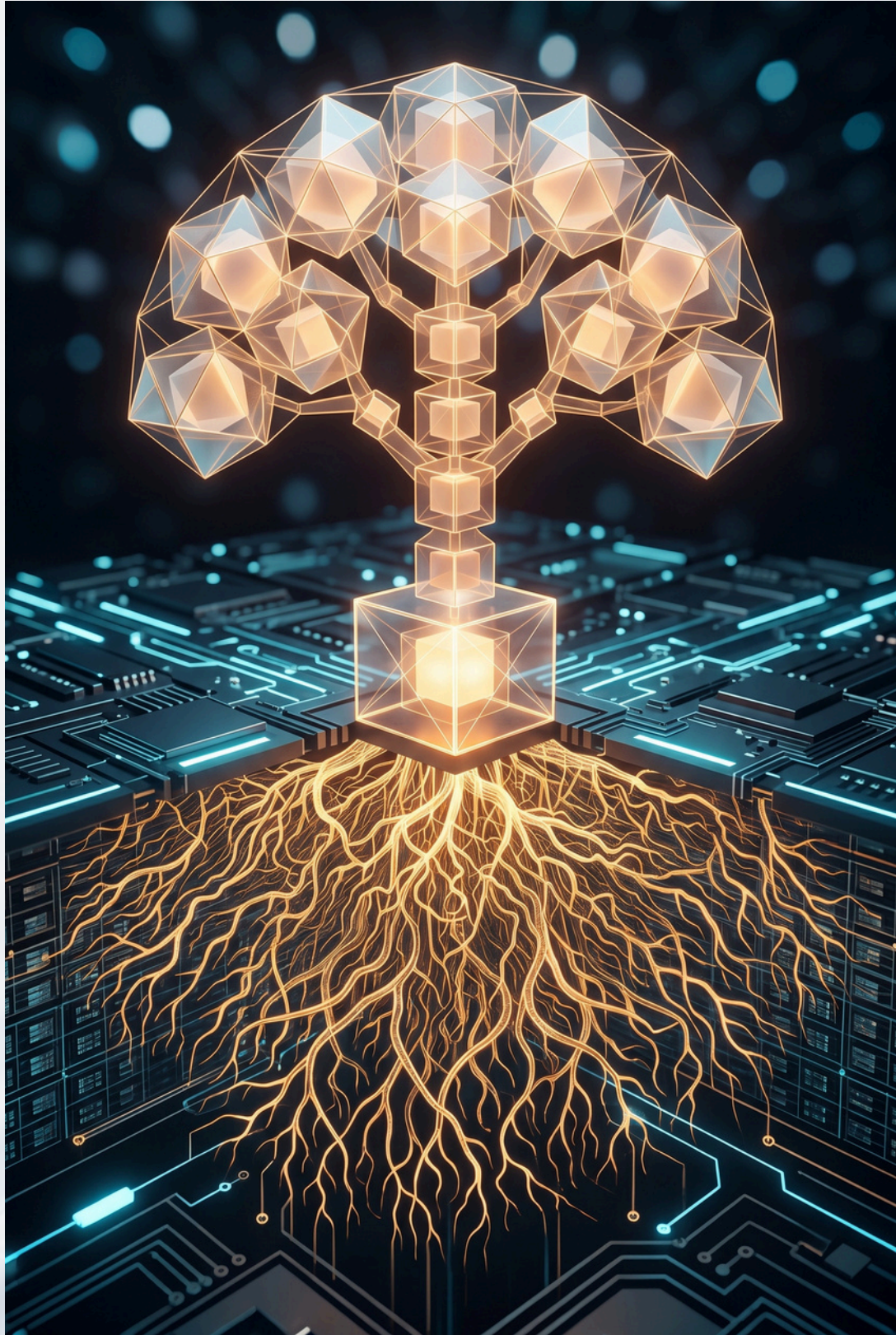
<https://docs.cloud.google.com/gemini-enterprise-agent-platform/govern/gateways/agw-partners>



Ping Identity integrates with Agent Gateway to deliver real-time, fine-grained authorization for agent and tool traffic. This integration ensures every request is continuously verified based on user, agent, context, and policy—moving beyond static credentials. Together, they provide centralized, consistent governance and visibility across all agent interactions without changing application code.

[Video](#)

[Blog](#)



AGENTIC SRE

Google AI SRE

SRE AI Autonomy Levels



	Monitor	Investigate	Mitigate	Actuate	Self-Direct
L0 – Manual	Automation	Human	Human	Human	Human
L1 – Assisted	Automation	Automation	Human	Human	Human
L2 – Partial	Automation	Automation	Human	Automation	Human
L3 – High	Automation	Automation	Automation	Automation	Human
L4 – Full	Automation	Automation	Automation	Automation	Automation

Google AI SRE whitepaper

The Safety Trifecta Google

Transparent AI reasoning - - - real-time risk assessments - - - progressive authorization

Strict architectural guardrails

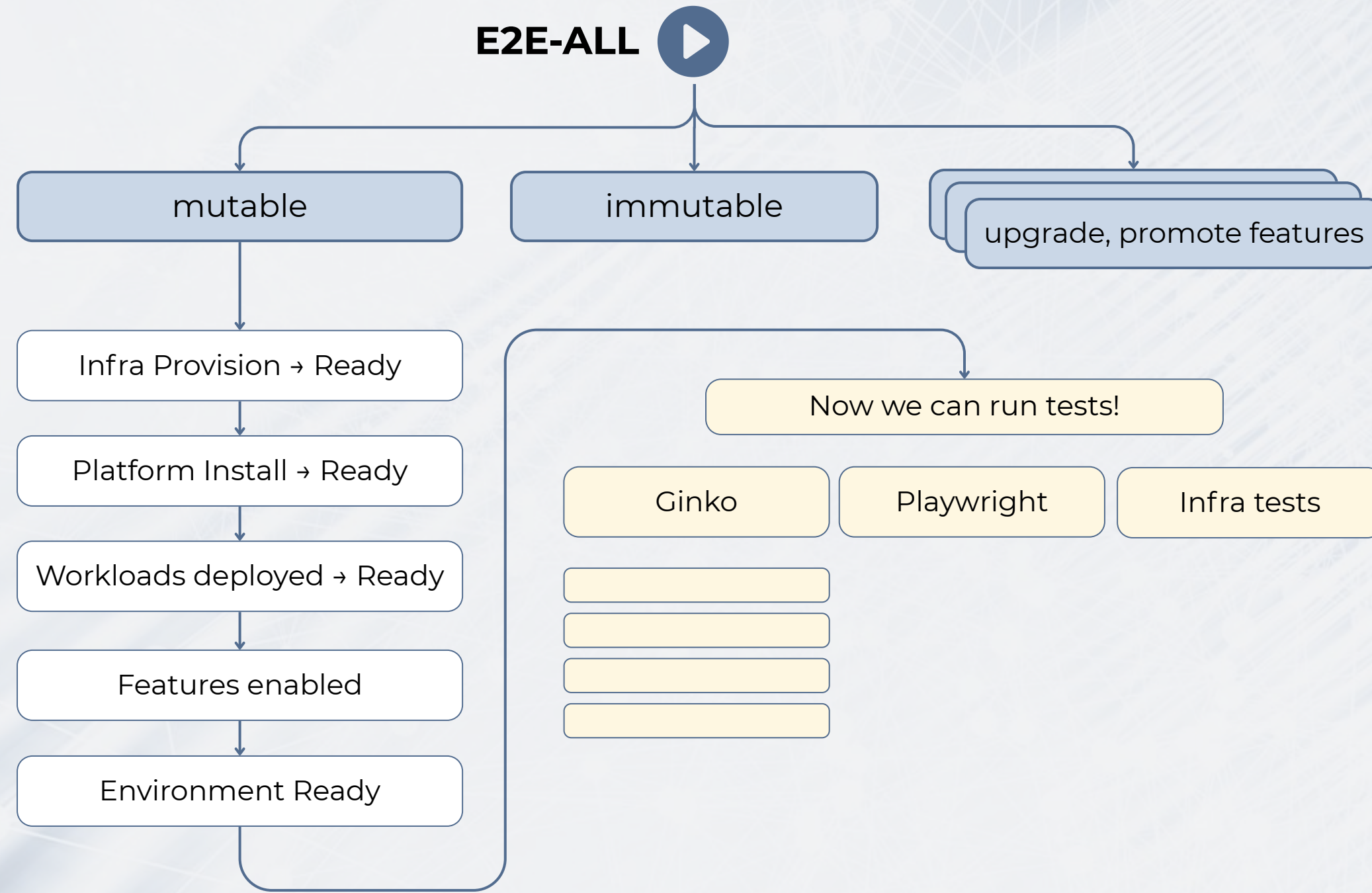
AI agents must operate under rigid technical constraints including least-privilege access, automated circuit breakers, zero-trust actuation tools.

Human Trajectories

Generating a human trajectory involves understanding and tracking the sequential actions and decisions made by engineers during an incident or a task. Creating and maintaining “Golden Trajectories” to feed evaluation pipelines.



First SRE Agent: e2e-agent



Multiple stages across multiple tests can fail

Wide range of error types – from infra to UI

Change rate is different for PR or release

Before e2e-agent

test failed
.....

Engineer's laptop

Engineer: /e2e-skill

AI: *Deliberating...*

Engineer: Follow-ups

AI: *Percolating...*



Engineer: Open ticket + implement

AI: *Booping...*

..... < days later >

Merged → Released → Resolved (?)

Time from test failure until investigation is unbound

Most failures go unaccounted or undiagnosed

Hands-on: requires Engineer's attention and time

Initial report time and quality can fluctuate

Most context stays on Engineer's laptop

Duplicate efforts by multiple Engineers

No full environment health overview

Transfer between timezone shifts is manual

Finding the middle ground

Deep analysis, correct for almost all failures

The ultimate goal is to be able to analyse correctly any e2e failure with high quality and depth of produced analysis.

Challenging agent structure, token management and identifying rightfully deep sessions from chasing-the-tail. Unpredictable execution time makes it harder for users to know when to expect the result.

Strictly bound time and token execution

Launch Phase, User-facing focus.

Provide reliable, sufficiently correct analysis for most e2e failures, grounded in evidence and source code, bounded by explicit time and token budgets and ready to guide following processes to final investigation and action.

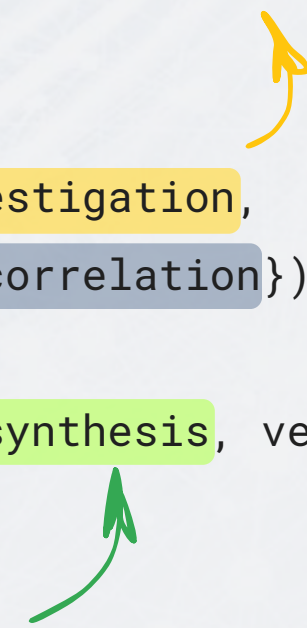
Trust earned through consistency



Agent Development Kit

Graph Workflows

```
root_agent = Workflow(  
    name="e2e-agent",  
    edges=[  
        ("START", intake, investigation, investigation_router),  
  
        (investigation_router,  
         {"continue": investigation,  
          "done": change_correlation}),  
  
        (change_correlation, synthesis, verifier, verifier_router),  
  
        (verifier_router,  
         {"repair": synthesis,  
          "done": done}),  
    ]  
)
```



```
investigator_agent = LlmAgent(  
    name="investigator",  
    instruction=INVESTIGATOR_INSTRUCTION,  
    tools=INVESTIGATOR_TOOLS .....  
)
```

Verifier

Rubber-stamp and a quality win

Overreach

log-only mechanism
claims; correlation !=
causation

Broken references

dangling references and
gap IDs; gaps
contradicting findings

Unverified claims

mechanisms with no
source code read

Report shape

no findings, placeholder
summary, malformed
verdict

+ a few
more.

Synthesis:

“Engine apply failure `serverside-applied ... signal: killed` the engine’s own
deadline killing the apply.... `client rate-limiter... context deadline
exceeded` - indicating API server saturation”

f_82bd4sd

certainty: proven

ev_28f7aa3

kind: log

Verifier

Rubber-stamp and a quality win

Overreach

log-only mechanism
claims; correlation !=
causation

Broken references

dangling references and
gap IDs; gaps
contradicting findings

Unverified claims

mechanisms with no
source code read

Report shape

no findings, placeholder
summary, malformed
verdict

+ a few
more.

Synthesis:

“Engine apply failure `serverside-applied ... signal: killed` the engine’s own
deadline killing the apply.... `client rate-limiter... context deadline
exceeded` - indicating API server saturation”

f_82bd4sd

certainty: proven

ev_28f7aa3

kind: log



`signal: killed` - external to the process

client rate limiter - mechanism to prevent
API server overload, not a symptom that
the server is actually overloaded.

Verifier:

“the logs show the symptoms; the causal mechanism is inferred - no
evidence of *kind code* to back it”

“no open gap recorded to confirm this claim.”

Verifier

Rubber-stamp and a quality win

Overreach

log-only mechanism
claims; correlation !=
causation

Broken references

dangling references and
gap IDs; gaps
contradicting findings

Unverified claims

mechanisms with no
source code read

Report shape

no findings, placeholder
summary, malformed
verdict

+ a few
more.

Synthesis:

“Engine apply failure `serverside-applied ... signal: killed` the engine’s own
deadline killing the apply.... `client rate-limiter... context deadline
exceeded` - indicating API server saturation”

f_82bd4sd

certainty: proven

ev_28f7aa3

kind: log



`signal: killed` - external to the process

client rate limiter - mechanism to prevent
API server overload, not a symptom that
the server is actually overloaded.

Verifier:

“the logs show the symptoms; the causal mechanism is inferred - no
evidence of *kind code* to back it”

“no open gap recorded to confirm this claim.”

Synthesis:

<revised analysis and recommendations>

f_82bd4sd

certainty: probable

Graph and Dynamic Workflows

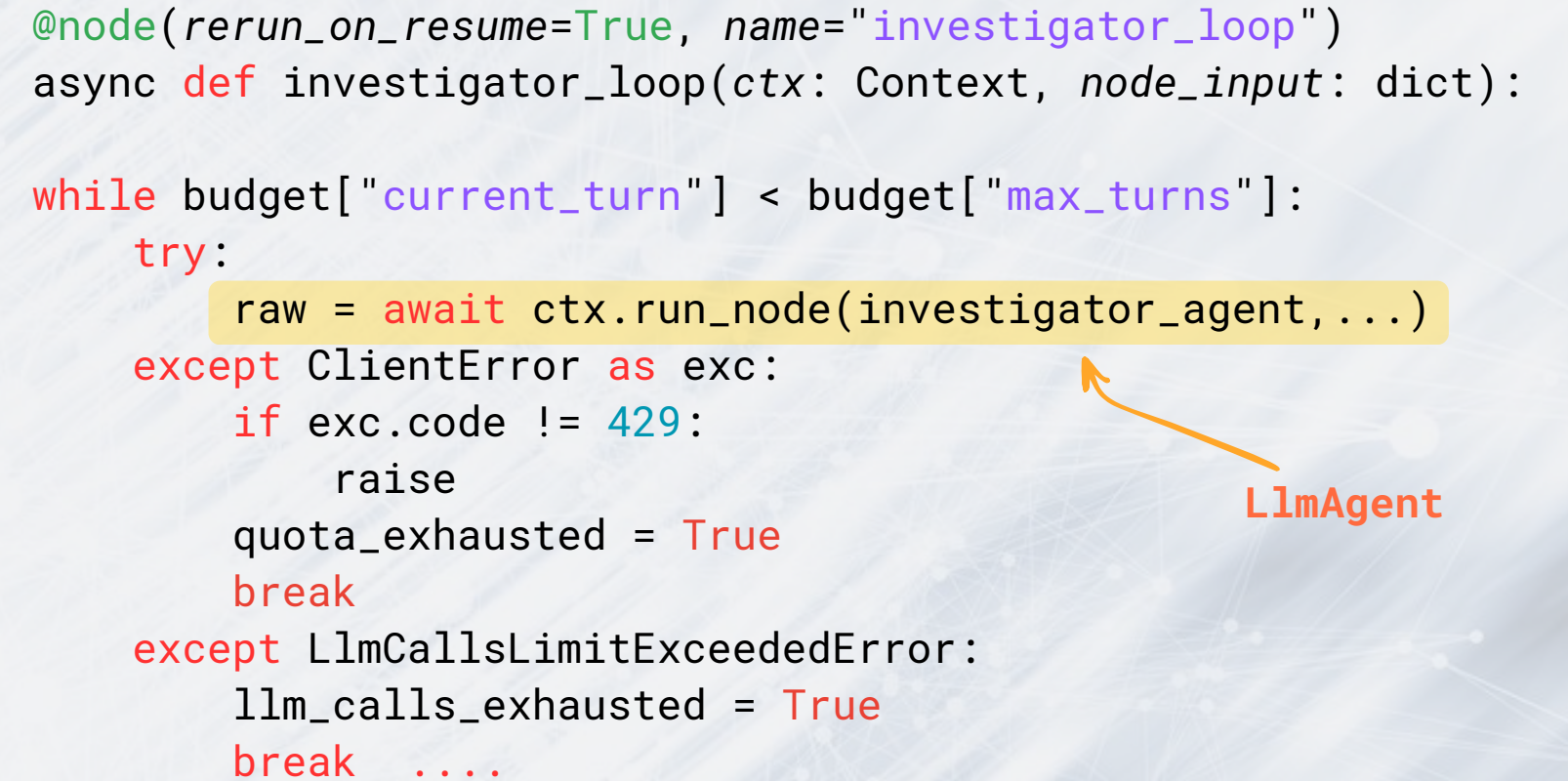
```
root_agent = Workflow(  
    name="e2e-agent",  
    edges=[  
        ("START", intake, investigation, investigation_router),  
        (investigation_router,  
         {"continue": investigation,  
          "done": change_correlation}),  
        (change_correlation, synthesis, verifier, verifier_router),  
        (verifier_router,  
         {"repair": synthesis,  
          "done": done}),  
    ]  
)
```

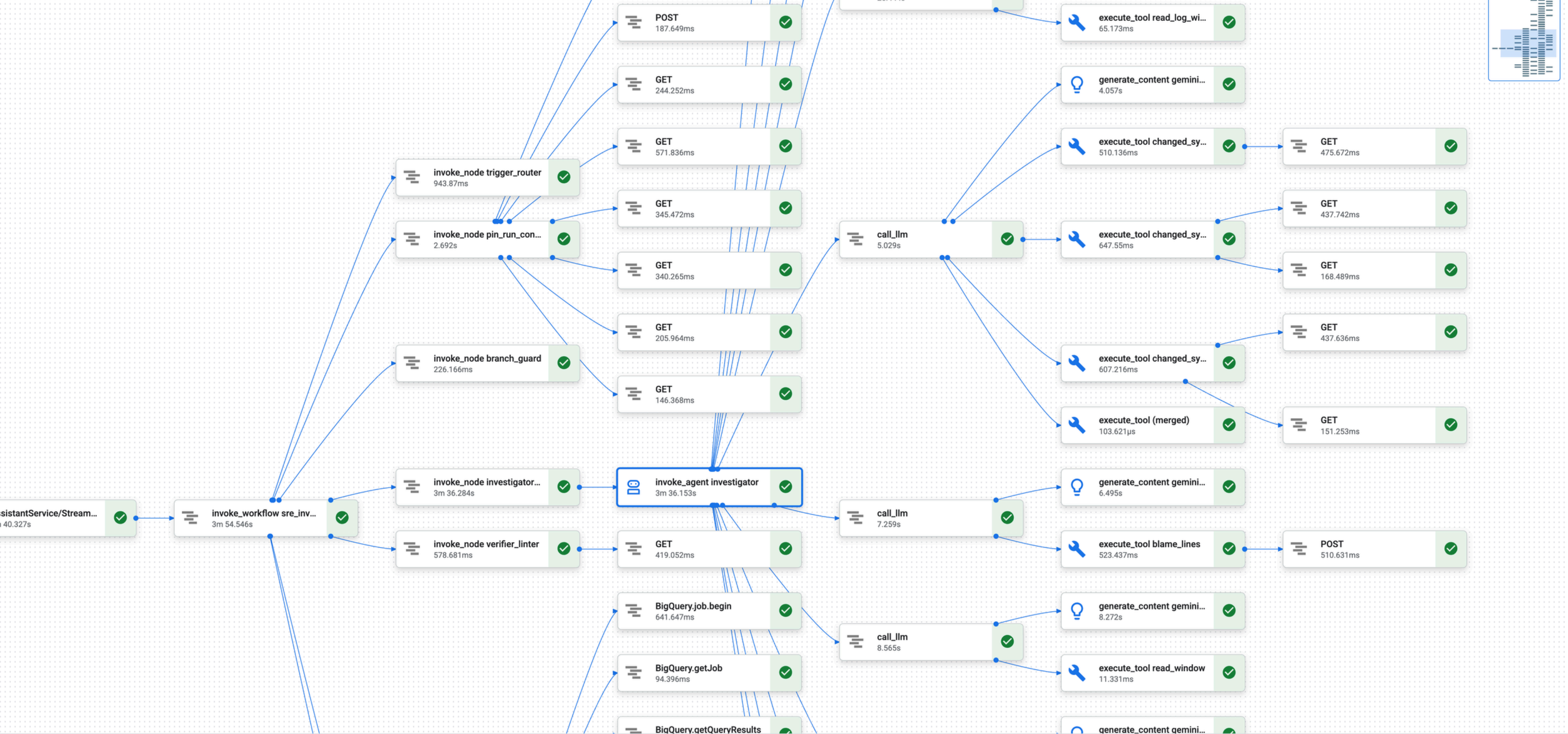


```
investigator_agent = LlmAgent(  
    name="investigator",  
    instruction=INVESTIGATOR_INSTRUCTION,  
    tools=INVESTIGATOR_TOOLS .....  
)
```

OR

```
@node(rerun_on_resume=True, name="investigator_loop")  
async def investigator_loop(ctx: Context, node_input: dict):  
    while budget["current_turn"] < budget["max_turns"]:  
        try:  
            raw = await ctx.run_node(investigator_agent,...)  
        except ClientError as exc:  
            if exc.code != 429:  
                raise  
            quota_exhausted = True  
            break  
        except LlmCallsLimitExceededError:  
            llm_calls_exhausted = True  
            break .....
```





138940336599728128 > invoke_agent investigator
3m 36.153s (77.1% of total time)

[Open session in Playground](#) [Copy](#) [View Logs](#) [X](#)

[Attributes](#) [Logs & Events](#) [Stacktraces \(0\)](#) [Metadata & Links](#)

Filter Enter property name or value		?	☒ GenAI attributes only
Key ↑	Value		
gen_ai.agent.description	GenAI		
gen_ai.agent.name	GenAI		
	investigator		

ADK: HITL Human-in-the-loop

Nested

```
hitl_workflow = Workflow(  
    name="sre_investigation"  
    edges=[  
        ("START", inner_workflow, dormancy_gate)  
        (dormancy_gate,  
         {"ask_deep_dive": targeted_deep_dive,  
          "close": session_complete }),....]
```

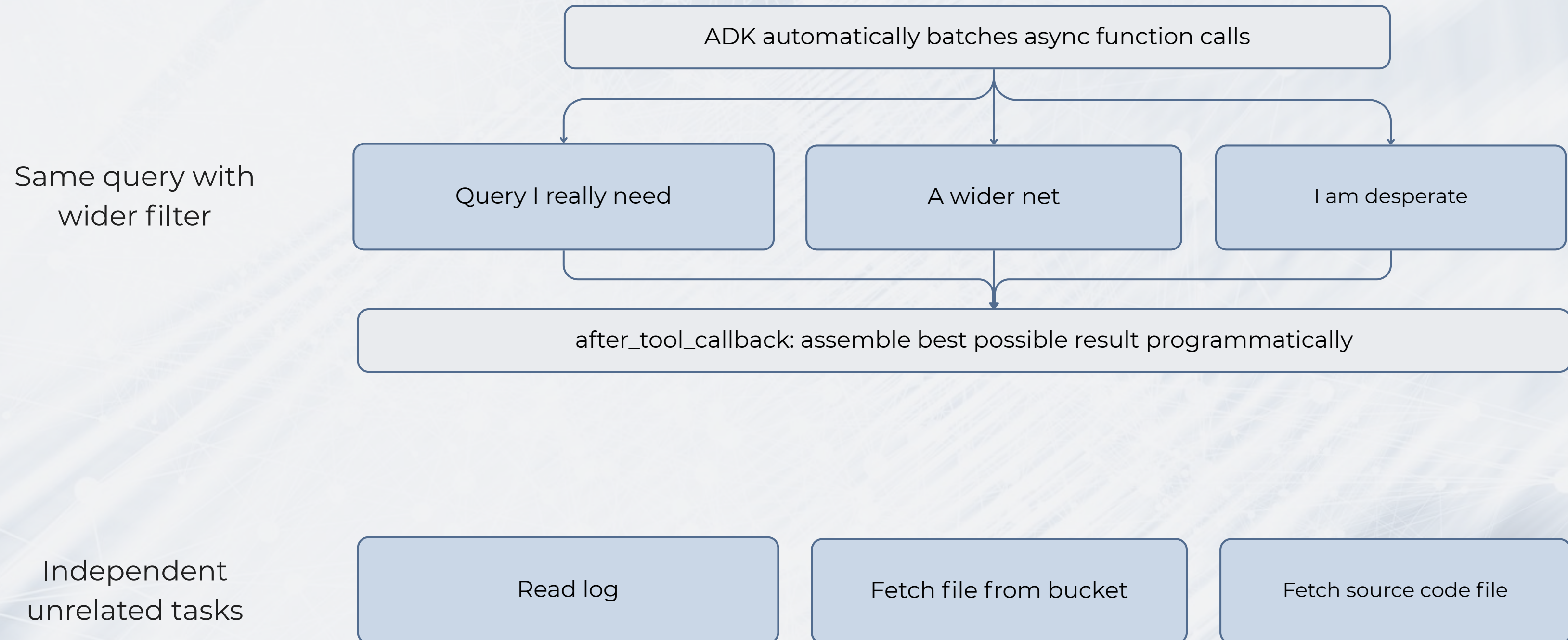
OR

Linear

```
hitl_workflow = Workflow(  
    name="sre_investigation",  
    edges=[  
        ("START", ..., router),  
        (router, {  
            "await_human": dormancy_gate,  
            "done": final_output,  
        }),  
        (dormancy_gate, {"invalid_feedback": dormancy_gate,  
                          "ask_deep_dive": targeted_deep_dive}),  
        (... , done)]
```

```
async def dormancy_gate(ctx: Context, node_input: dict):  
    yield RequestInput(  
        interrupt_id=interrupt_id,  
        message="Text for the user",  
        payload=node_input,  
        response_schema=DeepDiveFeedback  
    )
```

Parallel tool calls



go. To
ntext of

ations

be
e

nts,

AI is at the core of Agentic platform but it is

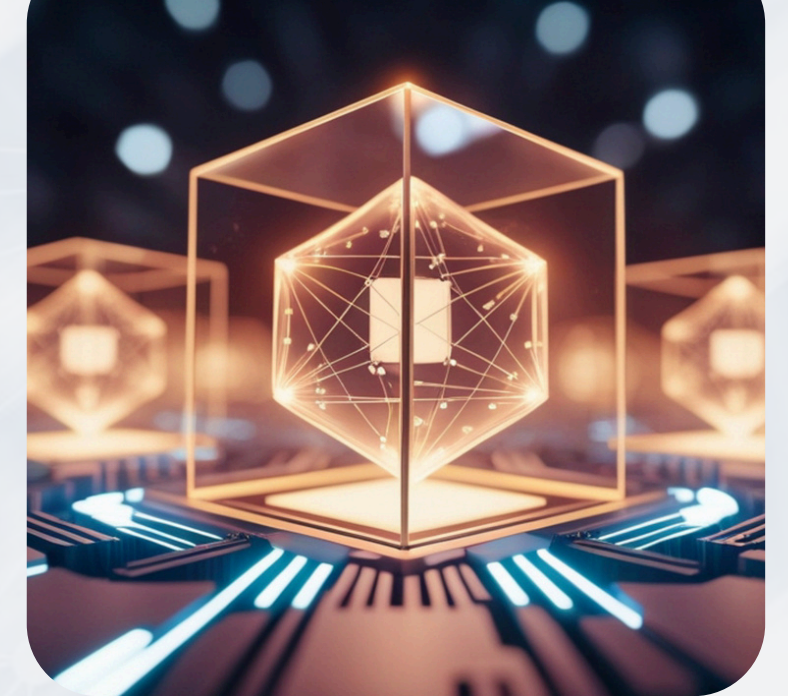
PRODUCT VISION & ARCHITECTURE

that define success



Product

Who are we building for?
How do we measure?



	Users of e2e	Environment Owners
What do they care about	My test run; Recommendations	Entire environment
Analysis depth	Just tell me if it is in my PR if so, where?	New issues, tracking → depth and accuracy is crucial
Coverage	single test run scope	All failures, attributed by release; defects lineage

Architecture

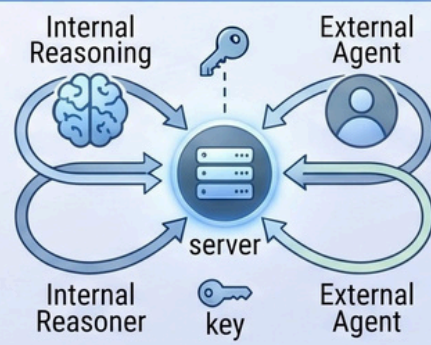
“Across all these four patterns none of them truly require bigger teams or newer models. They represent sound engineering practices that are frequently overlooked.”

4 BEST PRACTICES FOR BUILDING STRONG AI AGENTS

1. BI-DIRECTIONAL MCP

UNIFIED INTERNAL & EXTERNAL ACCESS

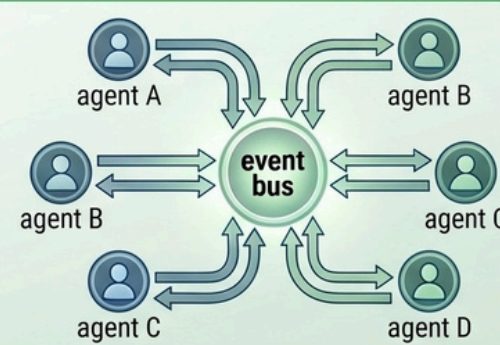
KEY VALUE:
Unified Context & Secure Access



2. EVENT-DRIVEN CONCURRENCY

ASYNC EVENT BUS

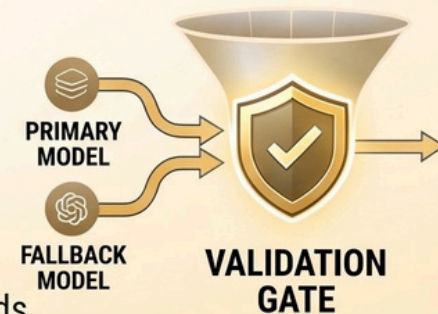
KEY VALUE:
Parallel Execution & Instant Signals



3. SAME-BAR FALLBACK

RIGOROUS OUTPUT VALIDATION

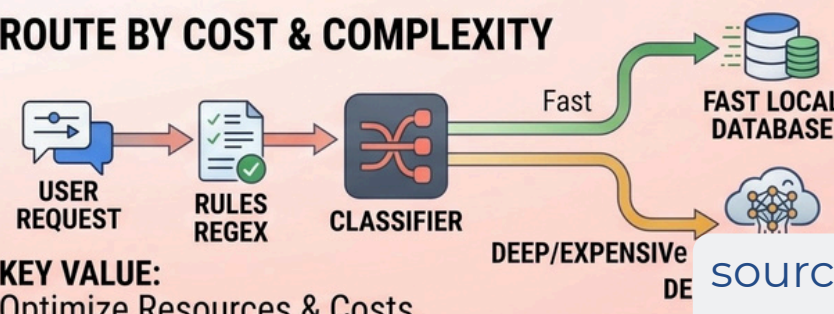
KEY VALUE:
Unified Quality & Standards



4. TIERED ROUTING

ROUTE BY COST & COMPLEXITY

KEY VALUE:
Optimize Resources & Costs



source:

<https://developers.googleblog.com/4-engineering-patterns-behind-the-strongest-ai-agents-challenge-submissions/>

The background is a dark, textured surface with a complex pattern of glowing orange and yellow lines that resemble a circuit board. These lines radiate from a central, bright, glowing square area. From this central point, numerous thin, white lines extend outwards, each ending in a small, glowing circle. The circles vary in size and brightness, creating a sense of depth and movement. The overall effect is one of high-tech, digital connectivity and energy.

DEMO

RECAP

Autonomy framework and gating conditions

Know which level your agent is at, and state what it must prove before it moves up.

Product and architecture

The model is the core, but knowing who you build for and applying sound engineering patterns is what decides success.

Structure over prompts

Budgets, routing and verification live in the graph and in code, not in instructions to the model.



REFERENCES / CONNECT

AI in SRE: How Google is Engineering the Future of Reliable Operations [whitepaper]

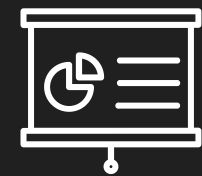
<https://sre.google/resources/practices-and-processes/ai-engineering-reliable-operations/>

Google for Startups AI Agents Challenge – winners, Sept. 2, 2026

<https://developers.googleblog.com/4-engineering-patterns-behind-the-strongest-ai-agents-challenge-submissions/>

ADK Coding with AI

<https://adk.dev/tutorials/coding-with-ai/>



olga-mir.github.io/



github.com/olga-mir/sre-agents



Olga Mirensky



github.com/olga-mir

